

EX.NO: 5

EXERCISE ON MODULES

DATE:

AIM:

To understand and implement JavaScript modules by creating reusable code components using export and import statements for better code organization and maintainability.

1. Create math.js exporting: add(a,b) subtract(a,b) multiply(a,b) Import all in main.js and calculate: add(5,3) subtract(10,4) multiply(2,6) Print the results.

PROGRAM:

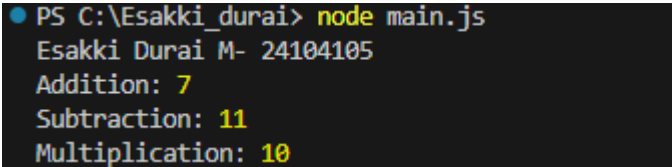
math.js:

```
export function add(a, b) {  
    return a + b;  
}  
export function subtract(a, b) {  
    return a - b;  
}  
export function multiply(a, b) {  
    return a * b;  
}
```

main.js:

```
import { add, subtract, multiply } from "./math.js";  
console.log("Esakki Durai M- 24104105");  
console.log("Addition:", add(5, 2));  
console.log("Subtraction:", subtract(19, 8));  
console.log("Multiplication:", multiply(2, 5));
```

OUTPUT:



```
PS C:\Esakki_durai> node main.js  
Esakki Durai M- 24104105  
Addition: 7  
Subtraction: 11  
Multiplication: 10
```

2. products.js: Export an array of 3 products: id, name, price. **cart.js:** Import products array from products.js. Write a function calculateTotal() to sum prices of all products. Export this function.

main.js: Import calculateTotal() from cart.js and print total value.

PROGRAM:

products.js:

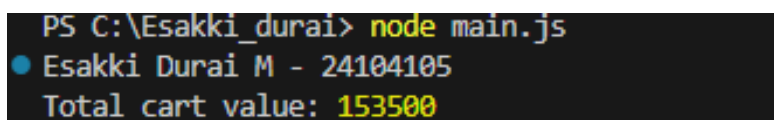
```
export const products = [
  {
    id: 1,
    name: "Laptop",
    price: 150000
  },
  {
    id: 2,
    name: "Mouse",
    price: 1000
  },
  {
    id: 3,
    name: "Keyboard",
    price: 2500
  }
];
```

cart.js

```
import { products } from "./products.js";
export function calculateTotal() {
  let total = 0;
  for (let product of products) {
    total += product.price;
  }
  return total;
}
```

main.js

```
import { calculateTotal } from "./cart.js";
console.log("Vigneshwara Baalan P - 24104065");
console.log("Total cart value:", calculateTotal());
```

OUTPUT:

```
PS C:\Esakki_durai> node main.js
● Esakki Durai M - 24104105
Total cart value: 153500
```

3. Write the students.js module with at least 3 students using module.exports. How will you use the built-in http module in server.js to create a server and display the student list? Extend students.js to include a function to add a new student, and explain how you will call it in server.js.

PROGRAM:

students.js:

```
const students = [
  {
    id: 1,
    name: "Vigneshwara Baalan",
    department: "CSE"
  },
  {
    id: 2,
    name: "Esakki Duari",
    department: "ECE"
  },
  {
    id: 3,
    name: "Kathick Selvam",
    department: "IT"
  }
];
function addStudent(student) {
  students.push(student);
}
module.exports = {
  students,
  addStudent
};
```

server.js

```
const http = require("http");
const { students, addStudent } = require("./students");
addStudent({
  id: 4,
  name: "Rahesh",
  department: "CSE"
});
const server = http.createServer((request, response) => {
  response.writeHead(200, {
    "Content-Type": "application/json"
  });

  response.end(JSON.stringify(students, null, 2));
});
server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT:

```
PS C:\Esakki_durai> node server.js
Server running at http://localhost:3000
```

Pretty-print ☐

```
[
  {
    "id": 1,
    "name": "Vigneshwara Baalan",
    "department": "CSE"
  },
  {
    "id": 2,
    "name": "Esakki Duari",
    "department": "ECE"
  },
  {
    "id": 3,
    "name": "Kathick Selvam",
    "department": "IT"
  },
  {
    "id": 4,
    "name": "Rahesh",
    "department": "CSE"
  }
]
```

The students.js file contains an array of student objects and an addStudent() function, both exported using module.exports. In server.js, the built-in http module is used to create a web server with http.createServer(). The students array and addStudent() function are imported using require(). By calling addStudent() before starting the server, a new student is added to the list, and when a user visits http://localhost:3000, the server responds by displaying the updated student list in JSON format.

Problem Undersatnding & Approach (10)	Implementation & Execution (15)	Tool/Technology Usage (10)	Code Quality & Documentation (5)	Viva (5)	Time Management (5)	Total (50)

RESULT:

Thus, JavaScript modules were successfully created to organize code into reusable components, demonstrating the use of export, import, module.exports, and require() statements.